

# Prose or Structure?

## Evaluating Security-Claim Generation in LLM-Based Code Analysis

Sebastian Grünewald

Diskussionsgrundlage für Dr. Andreas Ekelhart · 6. Oktober 2026

### 1 Idee und Ziel

Wie sollten Sprachmodelle die Ergebnisse einer Sicherheitsanalyse ausgeben, damit diese anschließend zuverlässig weiterverarbeitet werden können? Diese Arbeit vergleicht zwei Wege vom selben Quellcodekontext zu strukturierten Sicherheitsbehauptungen (*Claims*):

**P — über einen Report:** Code → Fließtextbericht → extrahierte Claims

**D — direkt strukturiert:** Code → direkt erzeugte Claims

Beide Wege sollen dasselbe Claimformat verwenden. Verglichen werden die Fundierung im bereitgestellten Code, die Abdeckung relevanter Aussagen sowie der Umgang mit Voraussetzungen und Unsicherheit. Nur bei P kommt die Treue zum ursprünglichen Report als zusätzliche Bewertungsgröße hinzu.

Ein Beispiel verdeutlicht das Problem: Aus „Bei aktivierter Exportfunktion könnten angemeldete Benutzer fremde Dateien lesen“ darf nicht ohne Weiteres „Jeder Benutzer kann fremde Dateien lesen“ werden. Die zweite Aussage lässt eine Bedingung weg und behauptet mehr Gewissheit. Umgekehrt kann eine fachlich falsche Aussage korrekt aus einem Report extrahiert sein. **Bedeutungstreue und tatsächliche Richtigkeit sind deshalb getrennte Fragen.**

Ziel ist ein überschaubares empirisches Paper über die *Erzeugung und Darstellung* von Security-Claims. Eine vollständige automatische Verifikationsplattform wird dafür zunächst nicht entwickelt.

### 2 Forschungsstand und vorläufige Abgrenzung

Die Grundidee, Aussagen zu zerlegen und einzeln zu prüfen, ist bereits erforscht. **DnDScore**, **DecompScore** und **VeriScore** untersuchen Zerlegung, Kontextbehandlung beziehungsweise überprüfbare Aussagen und deren Bewertung [1, 2, 3]. Im Softwarebereich zerlegt **LLMSAN** generierte Datenflusspfade in prüfbare Programmeigenschaften; **GPTAid** gewinnt Sicherheitsregeln unmittelbar aus API-Code und validiert sie dynamisch [4, 5]. **VERGE** und **EviGuard** verbinden strukturierte Claims mit unterschiedlichen Prüfverfahren [6, 7].

Der mögliche Beitrag liegt daher nicht in einer erstmaligen Claim-Zerlegung, einer neuen allgemeinen Claim-Sprache oder der bloßen Kombination von Verifiern. Untersucht werden soll vielmehr, welchen Unterschied der Report-Zwischenschritt bei einer *identischen Codegrundlage* und einem *gemeinsamen Zielschema* macht.

**Vorläufige Forschungslücke:** In den bisher betrachteten Arbeiten wurde kein unmittelbar entsprechender kontrollierter Vergleich identifiziert, der beide Wege anhand von Code-Fundierung, relevanter Abdeckung und erhaltenen Voraussetzungen bewertet. Diese Abgrenzung muss vor der Hauptstudie durch eine gezielte Prüfung der engsten Vorarbeiten weiter abgesichert werden. Eine Überlegenheit der direkten Route wird nicht vorausgesetzt.

### 3 Forschungsfragen

**RQ1 — Was muss ein Claim enthalten?** Welche Kontextangaben und Beziehungen sind erforderlich, damit Security-Claims eindeutig interpretiert und in passende Prüfaufgaben überführt werden können?

*Methode und Ergebnis:* Literatur und manuelle Kodierung von Entwicklungsfällen liefern ein begründetes Claimprofil samt Codebook und Grenzfällen.

**RQ2 — Welcher Erzeugungsweg liefert welche Qualität?** Wie unterscheiden sich P und D hinsichtlich fachlicher Fundierung, relevanter Abdeckung, angemessener Voraussetzungen und Aufwand?

*Methode und Ergebnis:* Gepaarter Vergleich mit unabhängigen Referenzinformationen. Ergebnis sind Qualitäts- und Kostenunterschiede samt Unsicherheit, kein vorab festgelegter Sieger.

**RQ3 — Welche Fehler wirken sich auf die Prüfung aus?** Welche Darstellungsfehler verändern die daraus abgeleitete Prüfaufgabe, und helfen explizite Kontextfelder, diese Fehler zu reduzieren?

*Methode und Ergebnis:* Fehleranalyse und eine kleine Ablationsstudie auf einem Teilbestand. Ergebnis ist eine Einordnung relevanter Fehler und ihrer Folgen; kein neuer allgemeiner Verifier.

### 4 Studiendesign und Claimformat

#### Daten und fairer Vergleich

Ausgangspunkt sind reproduzierbare Java-Fälle aus **Vul4J** mit Patches und Proof-of-Vulnerability-Tests [8]. Zunächst werden zwei bis drei passende Schwachstellenklassen gewählt. JSPWiki bleibt Entwicklungsfall; andere Fälle werden für die Evaluation zurückgehalten. Reparierte Varianten kontrollieren die konkrete Schwachstelle, sind aber nicht pauschal „sichere Programme“.

P und D erhalten identische Quellcode- und Konfigurationsdateien, aber keine Patches, PoVs, Advisories oder Benchmark-Labels. Bei P sieht der Extraktor nur den Report und führt keine zusätzliche Codeanalyse durch. Modellversion, Prompt, Kontext, Wiederholungen, Tokenbudget und Formatvorgaben werden dokumentiert.

Da P zwei Aufrufe benötigt und D zunächst einen, werden komplette Erzeugungswege verglichen. Eine zusätzliche D-Variante mit einem Überarbeitungsschritt sowie Kosten- und Laufzeitauswertungen dienen als Budgetkontrolle. Der Vergleich isoliert dadurch nicht automatisch den reinen Effekt des Ausgabeformats.

#### Ein kleines gemeinsames Claimprofil

CAE und SACM liefern die begriffliche Grundlage für Claims, Argumente und Evidenzbezüge [9, 10]. Für den Pilot genügt ein einfaches strukturiertes Format mit folgenden Feldgruppen:

| Feldgruppe               | Inhalt                                                                                                     |
|--------------------------|------------------------------------------------------------------------------------------------------------|
| Aussage                  | Claim-ID, Proposition, vorläufiger Claim-Typ.                                                              |
| Bedingungen und Umfang   | Akteur/Rechte, Voraussetzungen, Negation, Modalität, betroffene Version oder Konfiguration.                |
| Herkunft und Beziehungen | Codebezüge; Report-Textstelle bei P; explizit benannte verbundene Claims.                                  |
| Prüfbezug                | Benötigte Evidenz oder Prüfaufgabe, getrennt von der behaupteten Aussage und bereits vorliegenden Belegen. |

Annahmen sind keine bestätigten Systemzustände. Extrahierte Aussagen, ergänzender Kontext und abgeleitete Prüfaufgaben bleiben getrennt. Vollständige SACM-Implementierung und Lean-Formalisierung sind nicht vorgesehen.

## 5 Referenzbewertung und Auswertung

Es werden zwei Referenzen getrennt erstellt: Für P werden die tatsächlich im Report enthaltenen Aussagen und Bedingungen annotiert. Für beide Wege wird unabhängig davon festgehalten, welche relevanten Aussagen das bereitgestellte Codepaket rechtfertigt und welche Fragen offenbleiben. Ein echter CVE oder ein erfolgreicher PoV bestätigt nicht automatisch jede Aussage eines Reports.

Mindestens ein Teilbestand soll von zwei Personen unabhängig und möglichst ohne Kenntnis der Erzeugungsrouten annotiert werden. Unterschiedliche Bewertungen werden vor der gemeinsamen Klärung aufbewahrt. Ein zweites LLM kann unterstützen, bildet aber nicht allein die Referenz. Maßgeblich ist semantische Übereinstimmung, nicht identischer Wortlaut oder eine feste Anzahl von Claims.

| Kriterium               | Operationalisierung                                                                                                                    |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Fundierung im Code      | Anteil der durch den bereitgestellten Kontext belegten, widerlegten oder nicht entscheidbaren Aussagen; Unsicherheit separat bewerten. |
| Relevante Abdeckung     | Erfasste Propositionen eines vorab abgegrenzten Referenzbestands; keine Belohnung für bloß viele oder triviale Claims.                 |
| Bedeutungstreue (nur P) | Auslassungen, unbelegte Ergänzungen sowie Änderungen an Bedingungen, Negation, Akteur oder Aussagekraft.                               |
| Prüfbarkeit             | Eindeutige Entitäten, Geltungsbereiche und Voraussetzungen; Übereinstimmung der daraus formulierten Prüfaufgaben.                      |
| Aufwand und Robustheit  | Aufrufe, Tokens, Laufzeit, Kosten, Formatfehler und notwendige Reparaturen.                                                            |

Als Vergleich für die Extraktion in P dienen eine einfache satzweise Aufteilung, ein vorhandener allgemeiner Claim-Extraktionsansatz und die codebookgestützte Variante. D ist eine starke direkte Schema-Prompting-Baseline. Security-Systeme wie LLMSAN werden nur bei tatsächlich kompatiblen Teilaufgaben einbezogen. Kleine Zusatzversuche untersuchen die Wirkung expliziter Kontextfelder in beiden Routen.

Ausgewertet werden gepaarte Unterschiede pro Codefall mit Effektgrößen und Konfidenzintervallen. Claims desselben Falls sind keine unabhängigen Stichproben. Annotationsübereinstimmung, Ausfälle und offene Fälle werden separat berichtet. Ein nicht signifikanter Unterschied gilt nicht automatisch als Gleichwertigkeit.

## 6 Erarbeiteter Stand

### Gemeinsames Profil und Bewertungsgrundlage

Das **Claimprofil 0.1** liegt als Schema mit gemeinsamem Codebook vor und wird von beiden Routen verwendet. Es trennt die Proposition, sechs Kontextaspekte, Herkunft und Claimbeziehungen sowie die abgeleitete Prüfaufgabe. Nicht genannte Voraussetzungen werden von expliziter Negation und ausdrücklich unbekanntem Kontext unterschieden. Vorhandene Evidenz und Wahrheitsurteile gehören in die separate Annotation.

Grenzfälle wurden an JSPWiki sowie zwei weiteren Entwicklungsfällen ausgearbeitet. Diese Beispiele und die historische, assistierte JSPWiki-Annotation mit 13 Claims sind Entwicklungsmaterial; sie ersetzen keine unabhängige Referenz. Ein Annotationsprotokoll und neutrale Codepakete mit Arbeitsblättern sind vorbereitet. Die engsten Vorarbeiten wurden gezielt eingeordnet; eine abschließende Neuheitsprüfung steht noch aus.

Reproduzierbarer Forschungsprototyp

Ein kleiner Python-Prototyp deckt Fallvorbereitung, beide Erzeugungswege, Zusatzkontrollen und Annotationsexport ab. Fixierte Quellen und gemeinsame Formatierung stellen identische Codekontexte sicher. P verarbeitet sämtliche Findings eines Reports gemeinsam; D benötigt keinen erfolgreichen P-Report. Gemeinsame Validatoren prüfen Format und Beziehungen, bei P zusätzlich exakte Reportzitate. Sie entscheiden nicht über die inhaltliche Wahrheit.

Originaleingaben, Prompts, Modellantworten und Herkunft bleiben nachvollziehbar gespeichert. Ein budgetierter Versuchsablauf erfasst auch ungültige oder leere Ergebnisse, ohne sie automatisch zu reparieren oder zu wiederholen. 106 Offline-Tests prüfen unter anderem Kontexttrennung, Quellenpaarung und Fehlerbehandlung.

Erster Pilot: Erzeugung abgeschlossen, Bewertung offen

Der Pilot umfasst **fünf neue Vul4J-Fälle** aus zwei Familien: XXE (Apache CXF, OpenRefine) und Pfadverarbeitung (Plexus Archiver, Eclipse RDF4J, Retrofit). Je eine verwundbare und reparierte Variante ergeben zehn begrenzte Codekontexte. Jede Route wurde zweimal mit balancierter Reihenfolge ausgeführt. Alle Entwicklungs- und Pilotprojekte bleiben von der späteren zurückgehaltenen Evaluation ausgeschlossen.

Verwendet wurde gpt-5.4-mini-2026-03-17 mit deaktiviertem Reasoning. Hinzu kommen eine D-Revision je Kontext, die Kontextfeld-Ablation beider Routen auf vier Kontexten sowie Satz- und VeriScore-basierte Extraktionsbaselines. Die VeriScore-Baseline adaptiert den lizenzierten Originalprompt; sie ist keine Reproduktion des gesamten Systems. Die D-Revision kontrolliert einen zusätzlichen Aufruf, nicht ein identisches Tokenbudget.

| Bedingung                                | Ausgaben | Formal gültig |
|------------------------------------------|----------|---------------|
| P: Report / vollständige Claimextraktion | 20 / 20  | 20 / 16       |
| D: direkte Claims                        | 20       | 19            |
| D mit einem Überarbeitungsschritt        | 10       | 10            |
| Ohne explizite Kontextfelder: P / D      | 4 / 4    | 3 / 4         |
| Satz-/VeriScore-Baselinegruppen          | 4        | 4             |

Insgesamt wurden 82 Aufgaben mit 122 Modellaufrufen bearbeitet; sechs Ausgaben scheiterten an der Validierung. Die Fehler betreffen Zitatübereinstimmung, Subtypzuordnung, Zeilenangaben oder ein unerwartetes Feld. Rohantworten und Fehler sind erhalten. Der gesamte Pilot verursachte nach gemeldeten Tokens geschätzte API-Kosten von rund **1,09 USD**, einschließlich Zusatzkontrollen und ungültiger Ausgaben.

**Aussagekraft:** Der Pilot belegt die technische Durchführbarkeit und zeigt konkrete Ausgabeprobleme. Code-Fundierung, Abdeckung, Bedeutungstreue und Annotationsaufwand sind noch nicht bewertet. Formale Gültigkeit erlaubt keinen Qualitätsvergleich oder Schluss auf eine überlegene Route. Öffentliche Benchmarkkenntnis und die gezielt ausgewählten, begrenzten Codekontexte bleiben Limitationen. Neue Java-/PoV-Ausführungen wurden nicht vorgenommen.

## 7 Nächste Schritte und Abstimmung

1. **Unabhängige Referenzen erstellen.** Zuerst Codereferenzen allein anhand der bereitgestellten Codepakete, vor Einsicht in Modelloutputs. Danach die tatsächlichen P-Reporte unabhängig von den Extraktionen annotieren. Vorwissen und echte aktive Annotationszeit festhalten.
2. **Pilot bewerten und Fehler einordnen.** Claims beider Routen nach dem Codebook beurteilen. Für vier Kontexte (beide Varianten von CXF und Plexus Archiver) ist eine zweite unabhängige menschliche Annotation vorgesehen; Abweichungen werden anschließend gemeinsam geklärt. Technische Ausfälle und im Codekontext nicht entscheidbare Claims getrennt behandeln.
3. **Vergleichbarkeit und Aufwand prüfen.** Relevante Abdeckung, Code-Fundierung, P-Reporttreue und abgeleitete Prüfaufgaben gemeinsam mit vollständigen Routenkosten auswerten. Revision und Kontextablation als eigene Kontrollen betrachten. Varianten und Wiederholungen zunächst innerhalb des Falls zusammenführen.
4. **Hauptstudie festlegen.** Auf Grundlage des annotierten Piloten Umfang, zurückgehaltene Fälle, Fallzahlbegründung, Zeitplan und statistische Auswertung abstimmen. Literaturabgrenzung vervollständigen, anschließend Schema, Prompts, Annotationsregeln und Auswertungsplan einfrieren. Änderungen nach Pilotergebnissen explizit begründen.

**Für die Abstimmung mit Andreas:** Ist der Fokus auf den P/D-Vergleich für ein erstes Paper angemessen? Wer kann die unabhängige Zweitannotation übernehmen? Welcher Umfang der Hauptstudie ist nach dem Pilot realistisch, und wie sollen Referenzen und Forschungsartefakte zugänglich gemacht werden?

**Angestrebter Beitrag:** Ein nachvollziehbares Codebook, ein annotierter Referenzbestand und eine reproduzierbare Vergleichsstudie mit konkreten Befunden zu Informationsverlust, unberechtigten Ergänzungen und Qualitäts-/Aufwandskonflikten. Positive Ergebnisse werden ebenso wenig vorausgesetzt wie zwei Veröffentlichungen.

**Umfang:** Keine vollständige Agentenplattform, kein universeller Security-Scanner und kein eigenes Modelltraining. Eine mögliche Anschlussarbeit untersucht später, welche unabhängige Evidenz welche Claim-Typen tatsächlich rechtfertigt. Zunächst steht die Bewertung der erzeugten Claims im Mittelpunkt.

**Dokumentstatus.** Aktualisierte Diskussionsgrundlage für ein erstes Paper, kein formales Exposé einer gesamten Dissertation. Die Abschnitte 1–5 übernehmen den Projektplan der Skizze vom 03.10.2026; die Abschnitte 6–7 ersetzen den damaligen Arbeitsstand. Der Pilot ist technisch abgeschlossen, die empirische Qualitätsbewertung und die Festlegung der Hauptstudie stehen aus.

**Material.** Profil, Codebook, Versuchsprotokoll, technische Pilotübersicht und Quellcode: [github.com/sebi-gr/prose-or-structure](https://github.com/sebi-gr/prose-or-structure). Die Rohdaten des Piloten sind bislang lokal erhalten; ein freigegebenes Forschungsdatenarchiv steht noch aus.

## Ausgewählte Literatur

- [1] M. Wanner, B. Van Durme und M. Dredze (2025). *DnDScore: Decontextualization and Decomposition for Factuality Verification in Long-Form Text Generation*. EMNLP, S. 23609–23626. doi:10.18653/v1/2025.emnlp-main.1205.
- [2] M. Wanner, S. Ebner, Z. Jiang, M. Dredze und B. Van Durme (2024). *A Closer Look at Claim Decomposition*. Verwendete Fassung: arXiv:2403.11903. <https://arxiv.org/abs/2403.11903>.
- [3] Y. Song, Y. Kim und M. Iyyer (2024). *VeriScore: Evaluating the Factuality of Verifiable Claims in Long-Form Text Generation*. Findings of EMNLP, S. 9447–9474. doi:10.18653/v1/2024.findings-emnlp.552.
- [4] C. Wang, W. Zhang, Z. Su, X. Xu und X. Zhang (2024). *Sanitizing Large Language Models in Bug Detection with Data-Flow*. Findings of EMNLP, S. 3790–3805. doi:10.18653/v1/2024.findings-emnlp.217.
- [5] J. Liu, Y. Yang, K. Chen und M. Lin (2025). *Generating API Parameter Security Rules with LLM for API Misuse Detection*. NDSS. doi:10.14722/ndss.2025.230465.
- [6] V. Singh, D. Cassel, N. Weir, N. Feng und S. Bayless (2026). *VERGE: Formal Refinement and Guidance Engine for Verifiable LLM Reasoning*. Verwendete Fassung: arXiv:2601.20055v2. <https://arxiv.org/abs/2601.20055v2>.
- [7] H. Zhou, H. Lei und M. Yang (2026). *EviGuard: Machine-Verifiable Evidence Grounding for LLM-Based Industrial Incident Reasoning*. Applied Sciences, 16(18), 8925. doi:10.3390/app16188925.
- [8] Q.-C. Bui, R. Scandariato und N. E. Díaz Ferreyra (2022). *Vul4J: A Dataset of Reproducible Java Vulnerabilities Geared Towards the Study of Program Repair Techniques*. MSR, S. 464–468. doi:10.1145/3524842.3528482.
- [9] Adelard (o. J.). *Claims, Arguments and Evidence (CAE)*. Konzeptionelle Dokumentation; abgerufen am 03.10.2026. <https://www.adelard.com/asce/cae/>.
- [10] Object Management Group (2021). *Structured Assurance Case Metamodel (SACM), Version 2.2*. Normative Spezifikation und Metamodell. <https://www.omg.org/spec/SACM/2.2>.

**Redaktioneller Hinweis.** Der GPTAid-DOI wurde gegenüber der ursprünglichen Skizze bibliographisch korrigiert. Die ursprüngliche PDF bleibt unverändert.

**Layout.** LaTeX-Neusatz in Anlehnung an die Projektskizze; deren Layout ist eine vereinfachte Anpassung von „Template Text – University of Vienna“ (Universität Wien, CC BY 4.0). Ohne Logo und separate Titelseite; keine offizielle Research-Proposal-Vorlage.